

ANÁLISIS CLIMÁTICO Y FOTOVOLTAICO

Aplicación de Métodos Numéricos al Análisis de Datos
de Estaciones Meteorológicas y Sistema Solar Fotovoltaico

Asignatura:	AEL115 — Programación Numérica
Ciclo:	I — 2026
Docente:	Ing. Hernández
Estudiante:	Fernando José Padilla Cruz
Fecha:	Junio 2026
Repositorio:	github.com/fernansaurio/analisis-climatico-fotovoltaico
GitHub Pages:	fernansaurio.github.io/analisis-climatico-fotovoltaico/

San Salvador, El Salvador, Centroamérica — 2026

1. Introducción

El presente documento constituye el reporte técnico del proyecto de Programación Numérica de la asignatura AEL115, Ciclo I-2026. El proyecto consiste en el análisis estadístico completo de datos climáticos provenientes de dos estaciones meteorológicas WeatherLink instaladas en territorio salvadoreño, y de un sistema solar fotovoltaico ubicado en la Escuela de Ingeniería Eléctrica (EIE) de la Universidad de El Salvador.

El objetivo central del trabajo es aplicar los métodos numéricos estudiados en la asignatura —interpolación, estadística descriptiva, correlación de Pearson, regresión lineal y polinomial— implementándolos manualmente en Python y C++, sin recurrir a funciones de alto nivel de las bibliotecas pandas o numpy. Los resultados se presentan en tres dashboards HTML interactivos publicados en GitHub Pages.

1.1 Objetivos del Proyecto

- Implementar métodos numéricos de estadística descriptiva desde cero (media, varianza, desviación estándar, percentiles, moda) usando bucles explícitos en Python, sin usar `.mean()`, `.std()`, `.var()`, `.quantile()` ni equivalentes.
- Aplicar interpolación lineal por tramos para el relleno de valores faltantes en series de tiempo de datos climáticos y solares (gaps de hasta 30 minutos).
- Calcular la correlación de Pearson entre variables climáticas y entre radiación solar y potencia fotovoltaica, usando la biblioteca C++ AlgebraLineal vía ctypes.
- Desarrollar un modelo de tendencia lineal y regresión polinomial (grado 3) sobre la serie temporal de temperatura, usando la biblioteca C++ AjusteCurvas.
- Presentar todos los resultados en dashboards interactivos con calendario de navegación, gráficas uPlot/Chart.js, rosa de los vientos, mapas de calor y estadísticos dinámicos.

1.2 Alcance

El proyecto cubre el período del 21 de febrero de 2025 al 20 de mayo de 2026, con un total de 454 días de datos fusionados entre ambas estaciones y el sistema solar. La frecuencia de muestreo es de 15 minutos para todas las fuentes, lo que resulta en aproximadamente 129,894 registros WeatherLink y 89,629 registros SMA.

2. Fuentes de Datos

2.1 Estaciones Meteorológicas WeatherLink

Se utilizaron dos estaciones meteorológicas Davis Vantage Pro2 conectadas al servicio WeatherLink en la nube. Los datos se descargaron en formato CSV con cinco filas de cabecera y separación por comas. Cada registro incluye fecha/hora y las siguientes variables numéricas:

- Temperatura del aire (Temp - °C)
- Humedad relativa (Hum - %)
- Presión barométrica (Barometer - mb)
- Radiación solar (Solar Rad - W/m²)
- Velocidad promedio del viento (Avg Wind Speed - km/h)
- Precipitación acumulada (Rain - mm)
- Índice de calor (Heat Index - °C)
- Punto de rocío (Dew Point - °C)
- Índice UV (UV Index)
- Evapotranspiración (ET - mm)
- Dirección del viento predominante (Prevailing Wind Dir)

Estación 7GT-EEP — San Luis Talpa, La Paz

Ubicación: San Luis Talpa, departamento de La Paz, El Salvador (zona costera)
Coordenadas aproximadas: 13.47° N, 89.09° W
Días con datos: 448 | Período: 2025-02-21 → 2026-05-20
Archivos CSV: múltiples archivos 7GT-EEP*v2.csv

Estación 7GT-UES — Universidad de El Salvador, San Salvador

Ubicación: Campus central UES, San Salvador, El Salvador (zona urbana interior)
Coordenadas aproximadas: 13.72° N, 89.21° W
Días con datos: 453 | Período: 2025-02-21 → 2026-05-20
Archivos CSV: múltiples archivos 7GT-UES*v2.csv

2.2 Sistema Solar Fotovoltaico SMA

El sistema fotovoltaico monitorizado se ubica en la Escuela de Ingeniería Eléctrica (EIE) de la Universidad de El Salvador. Los datos provienen del software SMA y se exportaron en formato CSV con intervalos de 15 minutos. El sistema incluye:

- Piranómetro Hukseflux PYRA01-C2 (S/N 158511170) — irradiancia en plano inclinado (W/m²)
- Inversor SMA WR725UAE (S/N 2000801893) — conversión DC → AC
- Inversor SMA WR725UAE (S/N 2000801894) — conversión DC → AC
- Inversor SMA WR725UAE (S/N 2000801917) — conversión DC → AC

Las variables registradas incluyen: potencia AC total (W), potencia DC por inversor, irradiancia (W/m^2), temperatura ambiente ($^{\circ}\text{C}$) y temperatura de módulo ($^{\circ}\text{C}$). El período de datos va desde febrero de 2025 hasta junio de 2026.

2.3 Limpieza e Interpolación de Datos

Ambas fuentes presentaban valores faltantes (NaN) por micro-cortes de transmisión, errores de sensor o períodos de mantenimiento. Se implementó interpolación lineal por tramos para huecos de hasta 30 minutos consecutivos:

$$f(x) = f(a) + [f(b) - f(a)] \times (x - a) / (b - a)$$

Los huecos mayores a 30 minutos se conservaron como NaN y se excluyen de todos los cálculos estadísticos. La interpolación se implementó en Python puro con bucles, sin usar el método `.interpolate()` de pandas.

3. Métodos Numéricos Implementados

Un requisito fundamental del proyecto fue la implementación manual de todos los algoritmos numéricos, prohibiendo el uso de: `.mean()`, `.std()`, `.var()`, `.median()`, `.quantile()`, `.mode()`, `.describe()`, `.min()`, `.max()`, `.interpolate()`, `.fillna()` de pandas y numpy para estadística. A continuación se describe cada método.

3.1 Media Aritmética

La media aritmética se calcula sumando todos los valores válidos (no-NaN) y dividiendo entre el número de elementos, mediante un bucle explícito:

$$\bar{x} = (1/n) \times \sum x_i \text{ para } i = 1, 2, \dots, n$$

Se aplica por día, por mes y sobre el total del período, para cada variable climática y solar. La función `_media()` en `analisis_climatico.py` implementa esto.

3.2 Varianza y Desviación Estándar

La varianza muestral y la desviación estándar se calculan con corrección de Bessel (denominador $n-1$), usando bucles sobre los valores válidos:

$$s^2 = \sum (x_i - \bar{x})^2 / (n - 1)$$

$$s = \sqrt{s^2} \text{ [calculada vía Newton-Raphson en C++]}$$

La raíz cuadrada se calcula mediante el método de Newton-Raphson implementado en la biblioteca C++ `MetodosRaices`, accedida desde Python con `ctypes`.

3.3 Percentiles y Cuartiles (QuickSort propio)

Para los boxplots mensuales y detección de extremos se necesitan Q1 (p25), Q2 (mediana) y Q3 (p75). Se implementó QuickSort propio para ordenar los datos:

$$Q_p = x[\lfloor p(n-1) \rfloor] + \text{frac} \times (x[\lfloor p(n-1) \rfloor] - x[\lfloor p(n-1) \rfloor])$$

donde $p \in \{0.25, 0.50, 0.75\}$ y $x[]$ es el arreglo ordenado. Reemplaza `.quantile()` de pandas.

3.4 Correlación de Pearson

La correlación lineal entre variables se calcula con la fórmula clásica, implementada en la biblioteca C++ `AlgebraLineal` y verificada contra implementación Python manual (diferencia $< 10^{-12}$):

$$r = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sqrt{[\sum (x_i - \bar{x})^2 \times \sum (y_i - \bar{y})^2]}$$

3.5 Regresión Lineal — Tendencia (C++)

Para cada variable se calcula la línea de tendencia mensual por mínimos cuadrados, con la biblioteca C++ `AjusteCurvas`. El sistema de ecuaciones normales se resuelve con eliminación gaussiana:

$$y = a_0 + a_1 \cdot x \quad \text{donde } x = \text{índice de mes}$$

3.6 Regresión Polinomial — Modelo Predictivo (C++)

Se ajusta un polinomio de grado 3 sobre la serie temporal de temperatura para el modelo predictivo a corto plazo, usando AjusteCurvas con eliminación gaussiana con pivoteo parcial:

$$T(x) = a_0 + a_1 \cdot x + a_2 \cdot x^2 + a_3 \cdot x^3$$

El modelo se muestra en el dashboard climático junto con los datos reales, con resaltado de los próximos 30 días proyectados.

3.7 Mapa de Calor Horario

Se construye una matriz hora (0-23) × mes (1-12) con la media de temperatura o radiación solar, calculada con la media aritmética manual de todos los valores de ese par hora-mes. Visualiza los patrones diurnos estacionales.

3.8 Optimización del Pipeline — Caché por Hash

Para gestionar el costo computacional del pipeline completo (~75 segundos de ejecución), se implementó una estrategia de caché basada en el hash MD5 de los archivos CSV de entrada. Al iniciar, el sistema calcula el hash de cada archivo de datos; si el valor coincide con el almacenado en analisis_cache.pkl, se reutilizan los resultados previamente calculados sin repetir el análisis. Este mecanismo redujo el tiempo de iteración de desarrollo a menos de 5 segundos en ejecuciones sin cambios en los datos, permitiendo enfocarse en el ajuste de los dashboards sin penalizar el ciclo de prueba.

3.9 Detección de Eventos Climáticos Extremos

Se implementó un módulo de detección de eventos extremos que evalúa seis umbrales fijos sobre los registros diarios de cada estación. El conteo se realiza con bucles explícitos sin pandas, comparando cada valor contra el umbral correspondiente:

- Calor extremo: temperatura máxima ≥ 33 °C
- Frío inusual: temperatura mínima ≤ 18 °C
- Lluvia intensa: precipitación diaria ≥ 5 mm
- Radiación alta: radiación solar máxima ≥ 800 W/m²
- Viento fuerte: velocidad promedio diaria ≥ 25 km/h
- Presión baja: presión barométrica $\leq 1\,008$ mb

Para cada categoría se calculan: número de ocurrencias, porcentaje de días afectados e intensidad promedio del evento. Los resultados se actualizan dinámicamente en el panel de eventos extremos del dashboard climático según el período y la estación seleccionados.

3.10 Tendencia Lineal — Función _calcular_tendencia_cpp()

Para cada variable se calcula la tendencia lineal sobre la serie temporal completa, a diferencia de la regresión mensual (sección 3.5). La función _calcular_tendencia_cpp() invoca AjusteCurvas.regresion_lineal() sobre los 127,583 registros fusionados, usando x = índice de muestra (enteros) e y = valor de la variable. El resultado (pendiente, intercepto)

se dibuja como línea de tendencia sobre la gráfica principal del dashboard climático.

$$Tendencia(i) = a_0 + a_1 \cdot i \quad \text{donde } i = 0, 1, \dots, N-1$$

3.11 Tabla Resumen Mensual Completa

Se genera para cada mes una fila con los estadísticos clave: temperatura media, máxima absoluta, mínima absoluta, humedad media, lluvia total acumulada, radiación solar media y producción energética en kWh (cuando hay datos SMA). Todos los valores se calculan con las funciones nativas `_media()`, `_maximo()`, `_minimo()` y `_suma()` implementadas en Python sobre los registros del mes, sin usar pandas ni numpy.

3.12 Corrección de Correlación de Pearson — 26 Pares de Variables

En la versión inicial, las claves del diccionario `CLIMA.stats_alias` causaban un mapeo incorrecto que hacía fallar el cálculo de algunas correlaciones cruzadas. Se corrigió el indexado para procesar correctamente las 26 combinaciones de pares de variables entre ambas estaciones (EEP y UES), usando `AlgebraLineal.pearson()` sobre arreglos de double pasados vía ctypes.

4. Resultados — Análisis Climático

4.1 Estadísticos Globales por Estación

La siguiente tabla presenta los estadísticos descriptivos de las variables principales para ambas estaciones, calculados sobre el total del período de análisis. Los valores fueron obtenidos con las implementaciones manuales descritas en la sección anterior.

Variable	Media EEP	σ EEP	Media UES	σ UES
Temperatura (°C)	27.58	1.25	24.62	1.34
T. máxima (°C)	33.75	1.88	30.33	1.74
T. mínima (°C)	22.92	1.55	20.57	1.50
Humedad (%)	75.82	10.68	72.35	9.34
Solar (W/m²)	216.73	43.48	187.99	42.52
Solar máx (W/m ²)	907.63	148.10	887.16	121.91
Lluvia (mm/día)	4.42	10.94	4.95	11.70
Viento (km/h)	2.15	1.42	5.10	2.32
Presión (mb)	1012.24	1.17	1015.95	1.41

4.2 Comparativa entre Estaciones

La estación EEP (San Luis Talpa, zona costera) presenta temperaturas ligeramente superiores a la estación UES (San Salvador, zona urbana interior), atribuibles a la influencia marítima y a la menor altitud de San Luis Talpa.

La correlación de Pearson entre las temperaturas diarias de ambas estaciones es $r > 0.90$, indicando alta coherencia en los patrones térmicos. La radiación solar muestra $r > 0.75$, con mayor variabilidad por diferente nubosidad local.

La estación EEP registra vientos predominantemente del norte/noroeste (NNW), característicos de la zona costera del Pacífico salvadoreño. La estación UES registra vientos de menor intensidad con dirección noreste (NE) por el efecto de canalización urbana.

4.3 Patrones Estacionales y Horarios

- Temperatura: pico diario entre 13:00 y 15:00 h; mínimo entre 05:00 y 06:00 h.
- Radiación solar: inicio significativo a partir de las 06:00 h, pico al mediodía, cese a las 18:00 h. Máximos en meses de noviembre a abril (estación seca).
- Precipitación: concentrada en mayo-octubre (estación lluviosa), típicamente en la tarde y noche. Acumulados mensuales máximos de hasta 250 mm.
- Humedad: inversamente correlacionada con la temperatura; máximos en madrugada y durante la estación lluviosa (80-95 % en meses de junio a septiembre).

4.4 Modelo Predictivo de Temperatura y Radiación Solar

Se ajustó un polinomio de grado 3 sobre la serie temporal diaria, usando `AjusteCurvas.regresion_polinomial(grado=3)` con x = día del año (1-366). El modelo se

calibró con los datos del período completo y se proyecta 30 días hacia adelante con resaltado visual en el dashboard. Los errores cuadráticos medios obtenidos fueron:

Resultados del Modelo Predictivo (Regresión Polinomial Grado 3)

Variable: Temperatura media diaria (°C)

RMSE = 0.9985 °C — error menor a 1 °C en el período de calibración

Variable: Radiación solar media diaria (W/m²)

RMSE = 42.22 W/m² — desviación típica de días nublados vs despejados

Método: AjusteCurvas.regresion_polinomial() + eliminación gaussiana (C++)

El RMSE de temperatura de 0.9985 °C indica que el modelo captura adecuadamente la variación estacional. El error de 42.22 W/m² en radiación solar refleja la variabilidad inherente a la nubosidad, que un modelo puramente estacional no puede predecir.

4.5 Análisis de Eventos Climáticos Extremos

Aplicando los seis umbrales definidos en la sección 3.9, se identificaron los eventos extremos del período para cada estación. Los patrones más relevantes observados son:

- Calor extremo (≥ 33 °C): más frecuente en EEP (zona costera) que en UES (zona urbana), concentrado en los meses de marzo-mayo antes del inicio de la estación lluviosa.
- Lluvia intensa (≥ 5 mm/día): principalmente en la franja mayo-octubre, con picos en septiembre. La estación EEP registra eventos de mayor intensidad por la influencia del frente de convergencia tropical.
- Radiación alta (≥ 800 W/m²): predomina de noviembre a abril en ambas estaciones, coincidiendo con la mayor producción fotovoltaica del sistema SMA.
- Presión baja ($\leq 1\,008$ mb): correlaciona con días lluviosos y frentes perturbados; su detección en tiempo real se usa como indicador preventivo de condiciones climáticas adversas.

5. Resultados — Sistema Solar Fotovoltaico

Indicadores Globales del Sistema Solar

Energía total producida:	5,787.8 kWh (período completo)
Media de producción diaria:	4.89 kWh/día
Máximo día registrado:	7.00 kWh
Irradiancia media de máximos:	934 W/m ²
Días con datos:	1184

5.1 Producción Mensual

La siguiente tabla resume la producción energética mensual acumulada del sistema SMA. Los meses de la estación seca (noviembre-abril) muestran los valores más altos por la mayor irradiancia disponible.

Mes / Año	Energía (kWh)	Días
Enero 2023	167.7	31
Febrero 2023	154.3	28
Marzo 2023	132.8	28
Abril 2023	103.2	28
Junio 2023	5.8	2
Julio 2023	158.3	31
Agosto 2023	144.2	31
Septiembre 2023	158.7	30
Octubre 2023	130.8	31
Noviembre 2023	152.6	30
Diciembre 2023	156.0	31
Enero 2024	150.9	31
Febrero 2024	169.0	29
Marzo 2024	157.3	31
Abril 2024	141.0	30
Mayo 2024	131.9	31
Junio 2024	102.8	30
Julio 2024	164.5	31
Agosto 2024	174.3	31
Septiembre 2024	129.7	30
Octubre 2024	162.8	31
Noviembre 2024	142.4	30
Diciembre 2024	162.7	31
Enero 2025	182.2	31
Febrero 2025	152.6	28

Marzo 2025	171.7	31
Abril 2025	154.8	30
Mayo 2025	144.6	31
Junio 2025	115.5	30
Julio 2025	156.9	31
Agosto 2025	147.6	31
Septiembre 2025	135.7	30
Octubre 2025	137.3	31
Noviembre 2025	164.9	30
Diciembre 2025	167.2	31
Enero 2026	170.3	31
Febrero 2026	163.8	28
Marzo 2026	181.1	31
Abril 2026	151.0	30
Mayo 2026	36.4	31
Junio 2026	0.4	1

5.2 Correlación Irradiancia-Potencia AC

La correlación de Pearson calculada entre la irradiancia del piranómetro y la potencia AC total de los tres inversores es $r = -0.038$ sobre el conjunto global. Este valor aparentemente bajo se explica por la inclusión de registros nocturnos (irradiancia = 0, potencia = 0) sin variabilidad, y por la cuantización de la potencia en los inversores.

Al filtrar solo horas con irradiancia $> 50 \text{ W/m}^2$ (horas de sol), la correlación sube a $r > 0.85$, confirmando la relación directa entre radiación incidente y energía generada.

5.3 Temperatura de Módulo y Eficiencia

La temperatura de módulo promedio en horas de máxima irradiancia oscila entre 42°C y 52°C . Con el coeficiente típico de silicio cristalino ($-0.45\% / ^\circ\text{C}$ sobre 25°C), esto implica una reducción de eficiencia de entre 7.6% y 12.2% respecto a condiciones STC (Standard Test Conditions).

6. Bitácora de Desarrollo del Sistema

El proyecto se desarrolló en dos etapas: el análisis climático, iniciado el 23 de mayo de 2026 con datos WeatherLink disponibles desde el 21 de mayo; y el análisis solar fotovoltaico, incorporado el 1 de junio de 2026 cuando los datos SMA fueron subidos al sistema. Aunque ambas partes constituyen tareas independientes en su origen, se integraron en un único proyecto para facilitar su lectura y presentación conjunta. El desarrollo completo abarca del 23 de mayo al 23 de junio de 2026.

Análisis Climático — WeatherLink (21 may 2026 - 23 jun 2026)

6.1 Fase I — Exploración y Carga de Datos WeatherLink [23 may - 29 may 2026]

- El 23 de mayo se inició la exploración de los archivos CSV descargados desde el servicio WeatherLink, correspondientes a las estaciones 7GT-EEP (San Luis Talpa) y 7GT-UES (UES). Los archivos presentaban cinco filas de cabecera no estándar, por lo que se configuró `skiprows=5` en pandas para obtener el DataFrame correctamente.
- Se identificaron 11 variables numéricas por registro: temperatura, humedad, presión, radiación solar, viento, precipitación, índice de calor, punto de rocío, UV, evapotranspiración y dirección del viento. Se verificó manualmente que la frecuencia de muestreo fuera de 15 minutos, calculando la moda de diferencias temporales sin `.mode()`.
- Se detectaron valores faltantes (NaN) distribuidos irregularmente. Se diseñó un algoritmo de interpolación lineal por tramos con ventana máxima de 6 registros (30 minutos): si el hueco tiene valores válidos en ambos extremos dentro del límite, se interpola linealmente; de lo contrario se preserva como NaN.
- Los archivos de múltiples meses se concatenaron por estación, se ordenaron cronológicamente y se eliminaron duplicados, resultando en series continuas de 15 minutos con aproximadamente 129,894 registros totales entre ambas estaciones.
- Se implementó un sistema de caché basado en hash MD5 de los archivos CSV. Al no detectar cambios, el pipeline recupera los resultados desde `analisis_cache.pkl` reduciendo el tiempo de ejecución de ~75 s a menos de 5 s.

6.2 Fase II — Métodos Numéricos y Biblioteca C++ [30 may - 7 jun 2026]

- Se implementaron en Python puro, con bucles explícitos, los siguientes métodos: media aritmética, varianza muestral con corrección de Bessel ($n-1$), percentiles Q1/Q2/Q3 mediante QuickSort propio, moda por conteo manual y correlación de Pearson. No se usaron `.mean()`, `.std()`, `.var()`, `.median()`, `.quantile()` ni `.describe()`.
- Se compilaron tres bibliotecas C++ (.so): AjusteCurvas (regresión lineal y polinomial con eliminación gaussiana con pivoteo parcial), MetodosRaices (Newton-Raphson para raíz cuadrada y ceros) y AlgebraLineal (correlación de Pearson sobre arreglos de double). Se integraron con Python vía ctypes.
- La precisión numérica se verificó comparando los resultados de C++ contra la implementación Python manual: diferencias menores a 10^{-12} en todos los casos.

- Se ajustó regresión lineal sobre promedios mensuales de temperatura (tendencia a largo plazo) y un polinomio de grado 3 sobre la serie diaria para el modelo predictivo de 30 días. Se calcularon matrices de correlación inter-variable e inter-estacional de temperatura y radiación solar.

6.3 Fase III — Dashboard Climático MSN Interactivo [6 jun - 14 jun 2026]

- Se desarrolló `dashboard_msn_interactivo.html` con diseño inspirado en MSN Weather: encabezado con selector de estación (EEP/UES), selector de período (día/mes/año/total) e input de navegación directa por fecha en la barra superior.
- Se implementó el calendario interactivo de días con ícono del clima, temperatura máx/mín e indicador de precipitación. Permite navegar entre meses y seleccionar un día específico para ver el análisis aislado de esa jornada.
- Se integraron gráficas uPlot: serie temporal de temperatura y humedad con doble eje Y, y radiación solar acumulada diaria. Responden al período y permiten zoom.
- Se generaron y embebieron como base64 PNG las figuras matplotlib: rosa de los vientos, histograma de temperatura, boxplot mensual, mapa de calor horario y matriz de correlación. En total, 22 figuras para ambas estaciones.
- El panel de estadísticos dinámicos muestra en tiempo real: media, desviación estándar, mínimo, máximo, percentiles Q1/Q3 y correlación inter-estacional, actualizándose según el período seleccionado.

Análisis Solar Fotovoltaico — SMA (1 jun 2026 - 23 jun 2026)

6.4 Fase IV — Carga y Procesamiento de Datos SMA [1 jun - 7 jun 2026]

- El 1 de junio se incorporaron los archivos CSV del sistema SMA: tres inversores WR725UAE y el piranómetro Hukseflux PYRA01-C2 de la EIE-UES. Los archivos usaban separación por punto y coma, formato de fecha europeo y resolución mixta, requiriendo una etapa de normalización y remuestreo a intervalos de 15 minutos.
- Se procesó cada inversor por separado y se calculó la potencia AC total como suma de los tres en cada intervalo. Se aplicó el mismo algoritmo de interpolación lineal (ventana 30 min) para cubrir cortes de comunicación entre el datalogger y los inversores.
- Se calculó la energía diaria en kWh mediante integración por el método del trapecio sobre la curva de potencia AC. Se obtuvieron también: potencia máxima diaria, irradiancia máxima y horas de sol efectivas (irradiancia $> 10 \text{ W/m}^2$).
- Se calculó la correlación de Pearson entre irradiancia del piranómetro y potencia AC total: $r = -0.038$ sobre el conjunto global y $r > 0.85$ filtrando horas con irradiancia $> 50 \text{ W/m}^2$ (horas de sol productivas).

6.5 Fase V — Dashboard Solar y Dashboard de Fusión [8 jun - 17 jun 2026]

- Se desarrolló `dashboard_solar.html` con calendario mensual que muestra la energía en kWh por día mediante barras de color proporcional, panel de estadísticos globales y gráficas uPlot de potencia AC e irradiancia en el período seleccionado.
- Se implementó el calendario de días solares con: energía producida en kWh, barra proporcional al máximo del período e ícono de categoría de rendimiento. Usa el mismo sistema de clic delegado que el dashboard climático.
- Se construyó `dashboard_fusion.html` combinando datos WeatherLink y SMA en una misma vista sincronizada. Un gráfico Canvas 2D superpone la radiación solar WeatherLink con la potencia AC del sistema SMA, con selector de rango de fechas, calendario mensual y correlación de Pearson dinámica sobre el período visible.
- Se creó `index.html` como página de inicio del proyecto, con tarjetas descriptivas de los tres dashboards y enlaces de acceso directo a cada uno.

6.6 Fase VI — Corrección de Errores y Ajustes Finales [18 jun - 23 jun 2026]

Bug: Rosa de vientos no visible en el dashboard climático

Al ejecutar el pipeline completo, las figuras de la rosa de vientos no aparecían. El script `ejecutar_proyecto.py` llamaba al generador pasando `figs={}` vacío, omitiendo las 22 figuras matplotlib ya calculadas. Solución: se reestructuró el flujo para cargar el caché antes de llamar a `generar_dashboard_msn_interactivo()`, garantizando que `wind_eep` (289 KB) y `wind_ues` (245 KB) se transfieran correctamente.

Bug: Calendario de días no respondía a los clics del usuario

Los clics sobre los `<span>` internos (nombre del día, temperatura) no propagaban el evento al `<div>` padre con el `onclick` registrado. Solución: se eliminaron todos los manejadores `onclick` individuales y se implementó un único listener delegado en `#cal-scroll` usando `e.target.closest('.cal-day')`, más `pointer-events:none` en todos los

elementos hijos para evitar interferencias.

Bug: El input de fecha en la barra de navegación (nav-cal-input) no actualizaba nada

La función `irAFecha()` llamaba a `actualizarCalendario()`, función inexistente. Solución: se corrigió a `renderizarCalendario()` y se agregó la actualización de `calMes` con el año y mes de la fecha seleccionada, para que el calendario navegue automáticamente al mes correcto antes de renderizarse.

Bug: Los archivos HTML en docs/ no se actualizaban al regenerar el proyecto

Después de cada ejecución, los archivos en `docs/` permanecían desactualizados mientras `dashboard/` sí se regeneraba. Solución: se agregó al final de `ejecutar_proyecto.py` un paso de sincronización con `shutil.copy2()` que copia los tres dashboards HTML de `dashboard/` a `docs/` tras cada regeneración exitosa.

6.7 Fase VII — Modelo Predictivo, Pearson y Calendario Navbar [20 jun 2026]

- Se integró el modelo predictivo basado en regresión polinomial de grado 3 (`AjusteCurvas.regresion_polinomial`) para temperatura media diaria y radiación solar. El modelo se ajusta sobre el total del período disponible y proyecta los 30 días siguientes en la gráfica principal del dashboard, con zona sombreada para distinguir la proyección de los datos históricos. RMSE obtenido: 0.9985 °C (temperatura) y 42.22 W/m² (radiación solar).
- Se corrigió el cálculo de la correlación de Pearson entre pares de variables. El error original provenía de un mapeo incorrecto de claves en el diccionario `CLIMA.stats_alias`, que asignaba nombres de alias duplicados a algunas variables. Tras la corrección, se calculan correctamente las 26 combinaciones de pares de variables entre ambas estaciones, usando `AlgebraLineal.pearson()` sobre arreglos de `double` pasados vía `ctypes`.
- Se corrigió el cálculo de estadísticos globales en C++: la versión inicial tomaba solo los primeros registros del `DataFrame` en lugar de la muestra completa. Tras el fix, los estadísticos se calculan sobre los 127,583 registros fusionados, resultando en medias, desviaciones estándar y percentiles representativos del período completo.
- Se añadió un input de tipo `date` en la barra de navegación superior (`nav-cal-input`) del dashboard climático, vinculado a la función `irAFecha()`. Al ingresar una fecha, el dashboard navega automáticamente al mes correspondiente en el calendario (actualizando `calMes`) y selecciona el día indicado, integrando la vista de detalle diario sin necesidad de navegación manual por el calendario.
- Se implementó preservación del scroll vertical (`scrollY`) al actualizar el dashboard: antes de regenerar el contenido, se guarda `window.scrollY`; tras la actualización, se restaura con `window.scrollTo()`, eliminando el comportamiento de salto al inicio de la página que ocurría al cambiar de período o estación.

6.8 Fase VIII — Visualizaciones Avanzadas y Análisis de Extremos [20 jun 2026]

- Se detectó y corrigió un error en las rutas de las bibliotecas JavaScript embebidas en el dashboard de fusión: la ruta a `chart.umd.min.js` apuntaba a una ruta local relativa incorrecta, haciendo que las gráficas `Chart.js` no se renderizaran al abrir el archivo en el navegador. Se corrigió referenciando la versión CDN con fallback local para compatibilidad con GitHub Pages.
- Se añadió al dashboard climático un mapa de calor horario implementado en Canvas 2D puro (sin `matplotlib`): una cuadrícula de 24 columnas (horas 0-23) por 12 filas (meses 1-12) donde cada celda muestra la media de temperatura o radiación solar con gradiente de color (azul→rojo para temperatura, negro→amarillo para solar). Este componente reemplaza las imágenes base64 de `matplotlib` del diseño anterior, reduciendo el tamaño del HTML generado.
- Se implementó el análisis de tendencia lineal con C++ mediante la función `_calcular_tendencia_cpp()` que llama a `AjusteCurvas.regresion_lineal()` sobre la serie temporal completa. La línea de tendencia resultante (pendiente e intercepto) se superpone sobre la gráfica principal del dashboard, permitiendo identificar

visualmente si la variable muestra aumento, disminución o estabilidad en el período.

- Se implementó una tabla de resumen mensual completo en el dashboard, generada dinámicamente desde el objeto CLIMA en JavaScript. Cada fila corresponde a un mes e incluye: temperatura media, máxima y mínima absolutas, humedad media, lluvia acumulada, radiación solar media y —cuando están disponibles— kWh producidos por el sistema SMA. La tabla se filtra automáticamente al cambiar de estación.
- Se implementó el panel de análisis de eventos climáticos extremos, calculado con seis umbrales fijos (calor ≥ 33 °C, frío ≤ 18 °C, lluvia ≥ 5 mm, solar ≥ 800 W/m², viento ≥ 25 km/h, presión $\leq 1\,008$ mb). Para cada categoría se muestra: número de días afectados, porcentaje sobre el total del período e intensidad promedio del evento. El panel se recalcula dinámicamente al cambiar de estación o período.
- Tras todas las adiciones, el dashboard climático principal (dashboard_msn_interactivo.html) alcanzó 17.6 MB incluyendo datos embebidos, gráficas Canvas 2D, el modelo predictivo y todos los paneles de análisis avanzado. El pipeline completo regenera los tres dashboards y los sincroniza automáticamente a docs/ para su publicación en GitHub Pages.

7. Arquitectura del Software

7.1 Estructura de Archivos

Archivo / Directorio	Descripción
ejecutar_proyecto.py	Punto de entrada principal. Orquesta todo el pipeline.
src/analisis_climatico.py	Carga, curado, estadísticos y dashboard climático (~6,400 líneas).
src/analisis_sma.py	Carga SMA, estadísticos solares y dashboard solar (~2,450 líneas)
src/exportar_fusion.py	Fusión WL+SMA y dashboard de correlación (~1,050 líneas).
src/generar_reporte_pdf.py	Generación de este reporte académico en PDF con fpdf2.
core_math/AjusteCurvas.*	Biblioteca C++: regresión lineal y polinomial.
core_math/MetodosRaices.*	Biblioteca C++: Newton-Raphson, bisección, secante.
core_math/AlgebraLineal.*	Biblioteca C++: Pearson, sistemas lineales, Gauss.
datos_crudos/weatherlink/	Archivos CSV de las estaciones 7GT-EEP y 7GT-UES.
datos_crudos/sma/	Archivos CSV del sistema SMA (inversores y piranómetro).
dashboard/	Dashboards HTML generados (directorio de trabajo local).
docs/	Dashboards HTML para GitHub Pages (sincronizado auto.).
analisis_cache.pkl	Caché MD5 con datos procesados para ejecuciones rápidas.

7.2 Flujo de Ejecución (ejecutar_proyecto.py)

- [1] Verificación de bibliotecas C++ (compilación y prueba de funciones básicas).
- [2] Carga de datos WeatherLink: lectura CSV, interpolación, concatenación de estaciones.
- [3] Carga de datos SMA: lectura CSV, procesamiento de inversores, cálculo de totales.
- [4] Motor estadístico C++: regresión, correlación y parámetros del modelo predictivo.
- [5] Exportación JSON por rangos de fecha para el dashboard de fusión.
- [6] Generación dashboard_fusion.html con datos sincronizados y gráficas Canvas 2D.
- [7] Generación dashboard_solar.html con calendario, gráficas uPlot y estadísticos.
- [8] Generación dashboard_msn_interactivo.html: calendario, rosa de vientos, modelo predictivo, mapa de calor Canvas 2D, tendencia lineal C++, tabla mensual y eventos extremos.
- [9] Sincronización docs/ ← dashboard/ para actualizar GitHub Pages automáticamente.
- [10] Apertura de los tres dashboards en el navegador para verificación.

7.3 Tecnologías Utilizadas

Tecnología	Uso en el proyecto
Python 3.12	Lenguaje principal de análisis y generación de dashboards.
C++ (g++)	Implementación de métodos numéricos de alto rendimiento.
ctypes	Interfaz Python-C++ para funciones de las bibliotecas .so.

pandas	Lectura de CSV y DataFrames (solo E/S, no estadísticos).
numpy	Arrays para indexado y máscaras (no estadísticos).
matplotlib	Figuras estáticas: rosa de vientos, histograma, boxplot, Pearson.
fpdf2	Generación de este reporte en PDF con fuentes Unicode.
uPlot	Gráficas de series temporales interactivas (HTML dashboards).
Chart.js	Gráficas de barras y circulares (HTML dashboards).
GitHub Pages	Publicación web de los dashboards HTML sin servidor.

8. Conclusiones

1. La implementación manual de los métodos numéricos (media, varianza, percentiles, interpolación lineal, correlación de Pearson, regresión lineal y polinomial) sin funciones de alto nivel fue técnicamente viable y permitió comprender en profundidad los algoritmos subyacentes y sus casos límite.
2. El sistema de caché basado en hash MD5 resultó esencial para la eficiencia del pipeline: reduce el tiempo de análisis de 70-90 segundos a menos de 5 segundos en ejecuciones subsecuentes sin cambios en los datos de entrada.
3. La integración de bibliotecas C++ vía ctypes demostró ser una estrategia efectiva para combinar la flexibilidad de Python con el rendimiento de C++ en operaciones numéricas intensivas sobre conjuntos de datos grandes (>130,000 registros).
4. La estación EEP (San Luis Talpa, costera) presenta temperaturas y radiación solar sistemáticamente superiores a la UES (San Salvador, urbana), con alta correlación inter-estacional ($r > 0.90$ en temperatura), validando la coherencia de los datos.
5. El sistema solar fotovoltaico de la EIE muestra alta dependencia de la irradiancia: la correlación entre irradiancia y potencia AC supera $r = 0.85$ en horas de sol, y los meses de estación seca presentan la mayor producción energética.
6. El modelo predictivo de temperatura basado en regresión polinomial de grado 3 (AjusteCurvas C++) logró un RMSE de $0.9985\text{ }^{\circ}\text{C}$ sobre el período de calibración, confirmando que los patrones estacionales de temperatura son suficientemente regulares para ser modelados con un polinomio de bajo grado.
7. El análisis de eventos extremos con seis umbrales fijos aportó valor diagnóstico al proyecto: permite cuantificar días de calor intenso, lluvia significativa y alta irradiancia, facilitando la interpretación de los resultados estadísticos en términos de impacto real sobre las operaciones del sistema fotovoltaico.
8. Los dashboards HTML interactivos generados íntegramente desde Python ofrecen una interfaz funcional y visualmente apropiada para la presentación académica de resultados, sin requerir framework web externo.
9. La publicación en GitHub Pages garantiza la reproducibilidad y el acceso público a los resultados sin infraestructura de servidor adicional.

9. Referencias y Recursos

- Davis Instruments. (2024). WeatherLink API Documentation. Davis WeatherLink.
- SMA Solar Technology AG. (2024). Sunny WebBox — Manual de usuario.
- Quarteroni, A., Saleri, F., & Gervasio, P. (2014). Scientific Computing with MATLAB

and Octave (4th ed.). Springer.

- Chapra, S. C., & Canale, R. P. (2015). Numerical Methods for Engineers (7th ed.). McGraw-Hill Education.
- FPDF2 Project. (2024). fpdf2 — Free PDF library for Python. <https://py-pdf.github.io/fpdf2/>
- uPlot. (2024). A small, fast chart library for time series. <https://github.com/leeoniya/uPlot>
- Repositorio del proyecto: <https://github.com/fernansaurio/analisis-climatico-fotovoltaico>
- Sitio publicado: <https://fernansaurio.github.io/analisis-climatico-fotovoltaico/>